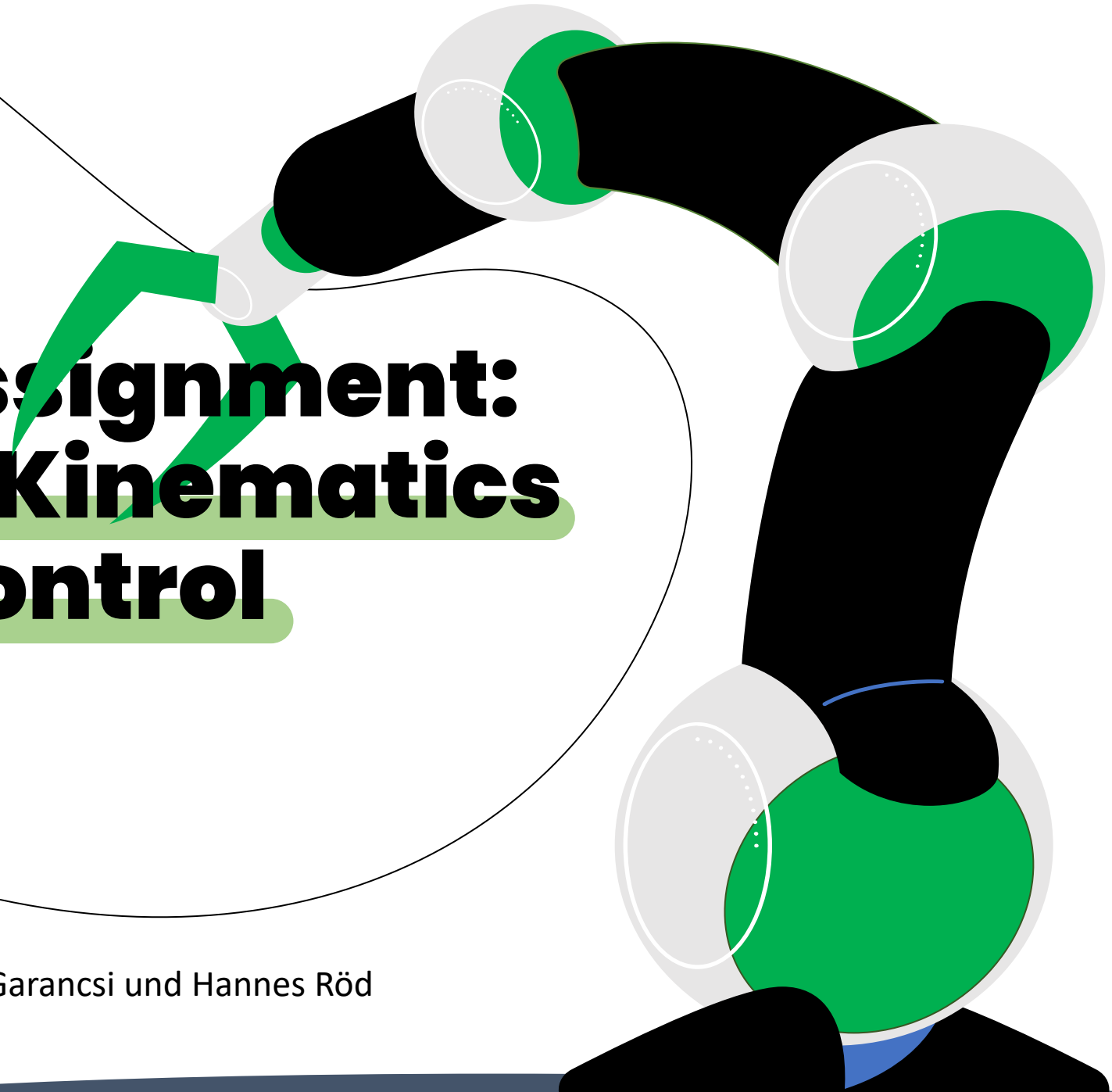




First Assignment: Inverse Kinematics Control

Leonhard Kohl-Lörting, Matyas Garancsi und Hannes Röd

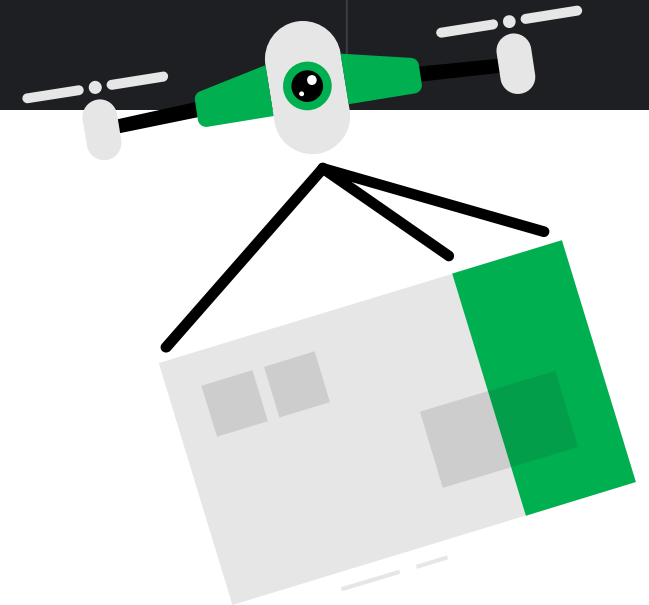
Initialisation:

- **First we initialized the Publisher and Subscriber**

```
# instantiate all the subscribers and publisher necessary
self.s_goal_euler = rospy.Subscriber('/Franka/goal_euler', EulerPose, self.euler_pose_callback)
self.p_goal = rospy.Publisher('/Franka/goal', Pose, queue_size=10)
self.p_end_effector = rospy.Publisher('/Franka/end_effector', Pose, queue_size=10)
for i in range(0, self.dof):
    self.s_joint_state[i] = rospy.Subscriber('/Franka/joint' + str(i+1) + '/state', Float64, self.joint_state_callback, callback_args=i)
    self.p_joint_cmd_vel[i] = rospy.Publisher('/Franka/joint' + str(i+1) + '/cmd_vel', Float64, queue_size=10)
    self.p_A[i] = rospy.Publisher('/Franka/A' + str(i+1), TransformationMatrix, queue_size=10)
    self.p_T[i] = rospy.Publisher('/Franka/T' + str(i+1), TransformationMatrix, queue_size=10)
    self.p_frame[i] = rospy.Publisher('/Franka/Frame' + str(i+1), Pose, queue_size=10)
self.p_jacobian = rospy.Publisher('/Franka/Jacobian', Jacobian, queue_size=10)
```

- **And read the parameters from the .launch file**

```
# read the server parameters
self.position_gain = rospy.get_param("/position_gain")
self.orientation_gain = rospy.get_param("/orientation_gain")
self.damping_coefficient = rospy.get_param("/damping_coefficient")
```



Forward Kinematics

- **We created two callback functions**

```
# callback function that is called when a new euler pose is published
1 usage  👤 Mátyás Garancsi +1
def euler_pose_callback(self, data):
    self.desired_pose_euler = data

# callback function that is called when a new joint state is published
1 usage  👤 admin +2
def joint_state_callback(self, data, number):
    # save the joint state in the corresponding position in the array
    self.joint_states[number] = data.data
```

- **Once all states were published we called the update function to calculate the forward kinematics**

```
self.relative_transformations = [self.get_homogeneous_transformation(i, joint_configuration)
                                for i in range(self.dof + 1)]
self.cumulative_transformations = self.get_cumulative_transformation(joint_configuration)
```

Forward Kinematics: Homogeneous Transformation

$$A_i^{i-1}(q_i) = A_{i'}^{i-1} A_i^{i'} = \begin{bmatrix} c_{\vartheta_i} & -s_{\vartheta_i} c_{\alpha_i} & s_{\vartheta_i} s_{\alpha_i} & a_i c_{\vartheta_i} \\ s_{\vartheta_i} & c_{\vartheta_i} c_{\alpha_i} & -c_{\vartheta_i} s_{\alpha_i} & a_i s_{\vartheta_i} \\ 0 & s_{\alpha_i} & c_{\alpha_i} & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

```
def get_homogeneous_transformation(self, n, joint_configuration: np.ndarray) -> np.ndarray:
    if n == 0:
        return np.eye(4)

    # extract the DH parameters
    a = self.dh_parameters[n - 1]['a']
    alpha = self.dh_parameters[n - 1]['alpha']
    d = self.dh_parameters[n - 1]['d']
    theta = self.dh_parameters[n - 1]['theta'] + joint_configuration[n - 1]

    # Calculate the transformation matrix
    T = np.array([
        [np.cos(theta), -np.sin(theta) * np.cos(alpha), np.sin(theta) * np.sin(alpha), a * np.cos(theta)],
        [np.sin(theta), np.cos(theta) * np.cos(alpha), -np.cos(theta) * np.sin(alpha), a * np.sin(theta)],
        [0, np.sin(alpha), np.cos(alpha), d],
        [0, 0, 0, 1]
    ])

    return T
```

Forward Kinematics: Cumulative Transformation

```
def get_cumulative_transformation(self, joint_configuration: np.ndarray) -> List[np.ndarray]:  
    """  
    Returns a list of  $T_0^0$ ,  $T_1^0$ ,  $T_2^0$ ,  $T_3^0$ , ...,  $T_e^0$   
    :param joint_configuration:  
    :return: list of homogeneous transformation matrixes  
    """  
  
    # initialize the list with the identity matrix  
    T = [np.eye(4)]  
  
    # compute the cumulative transformation matrixes  
    for i in range(1, self.dof + 1):  
        T.append(T[i - 1] @ self.relative_transformations[i])  
  
    return T
```

Inverse Kinematics Algorithm (Unit Quaternion)

Input: x_d, λ, K # des. vel, damp coeff, error weighting

while True:

$q_t = \text{read_robot_configuration}()$

$x_t = \text{forward_kinematics}(q_t)$

$e_t = \eta_e(q)\epsilon_d - \eta_d\epsilon_e(q) - S(\epsilon_d)\epsilon_e(q)$

$\dot{x}_t = K e_t$

$J_G = \text{compute_geometric_Jacobian}(q_t)$

$\dot{q} = (J_G^T J_G + \lambda I)^{-1} J_G^T \dot{x}_t$

$\text{send_velocities}(\dot{q})$

```
def get_elementary_ik(self, desired_position, desired_euler, damping_coefficient):
    current_pose = self.get_uq_pose()
    current_position = current_pose[0]
    current_orientation_uq = current_pose[1]

    desired_orientation_uq = self.get_uq_from_euler(np.array([desired_euler.phi, desired_euler.theta, desired_euler.psi]))

    error_orientation = current_orientation_uq.error(desired_orientation_uq)

    error_position = desired_position - current_position

    error = np.concatenate((error_position, error_orientation))
    x_t = np.dot(self.K, error)

    J = self.get_geometric_jacobian()

    # calculate damped pseudo-inverse with error
    J_inv = np.linalg.inv(np.dot(J.T, J) + damping_coefficient * np.eye(self.dof)) @ J.T

    # calculate the joint velocity
    q_dot = np.dot(J_inv, x_t)

    return q_dot
```

Error Function

$$e_t = \eta_e(q)\epsilon_d - \eta_d\epsilon_e(q) - S(\epsilon_d)\epsilon_e(q)$$

```
def error(self, desired_uq: UnitQuaternion) -> np.ndarray:  
    return self.eta * desired_uq.epsilon - desired_uq.eta * self.epsilon - np.cross(desired_uq.epsilon, self.epsilon)
```

$$\dot{x}_t = K e_t$$

```
self.K = np.diag([position_gains, position_gains, position_gains, orientation_gains, orientation_gains, orientation_gains])
```

Geometric Jacobian

$$J_G(q) = \left[\overbrace{\begin{matrix} z_0^o \times (p_e^o - p_0^o) \\ z_0^o \end{matrix}}^{J_1(q)}, \overbrace{\begin{matrix} z_1^o \times (p_e^o - p_1^o) \\ z_1^o \end{matrix}}^{J_2(q)} \right]$$

```
def get_geometric_jacobian(self) -> np.ndarray:
    jacobian = np.zeros((6, self.dof))
    for i in range(self.dof):
        # extract the z_{i-1} vector
        z = self.cumulative_transformations[i][0:3, 2]
        # extract the p_{i-1} vector
        p = self.cumulative_transformations[i][0:3, 3]

        # z_{i-1} x (p_e - p_{i-1})
        jacobian[0:3, i] = np.cross(z, self.cumulative_transformations[self.dof][0:3, 3] - p)
        # calculate the z_{i-1} vector
        jacobian[3:6, i] = z
    return jacobian
```

$$J_G(q) = \left[\begin{array}{cc} -s_{12}a_2 - s_1a_1 & -s_{12}a_2 \\ c_{12}a_2 + c_1a_1 & c_{12}a_2 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 1 & 1 \end{array} \right] \left. \vphantom{\begin{array}{cc} -s_{12}a_2 - s_1a_1 \\ c_{12}a_2 + c_1a_1 \\ 0 \\ 0 \\ 0 \\ 1 \end{array}} \right\} z_i^o$$

Publish Goal and Joint-Velocities

```
# publish the goal pose in unit quaternion
pose = Pose()
pose.position.x = self.desired_pose_euler.position.x
pose.position.y = self.desired_pose_euler.position.y
pose.position.z = self.desired_pose_euler.position.z
pose.orientation.x = goal_quat.epsilon[0]
pose.orientation.y = goal_quat.epsilon[1]
pose.orientation.z = goal_quat.epsilon[2]
pose.orientation.w = goal_quat.eta
self.p_goal.publish(pose)

# INVERSE DYNAMICS
# calculate velocity

geo_jacobian = self.robot_kinematics.get_geometric_jacobian()
desired_position = np.array([self.desired_pose_euler.position.x, self.desired_pose_euler.position.y, self.desired_pose_euler.position.z])
q_dot = self.robot_kinematics.get_elementary_ik(desired_position, self.desired_pose_euler.orientation, self.damping_coefficient)

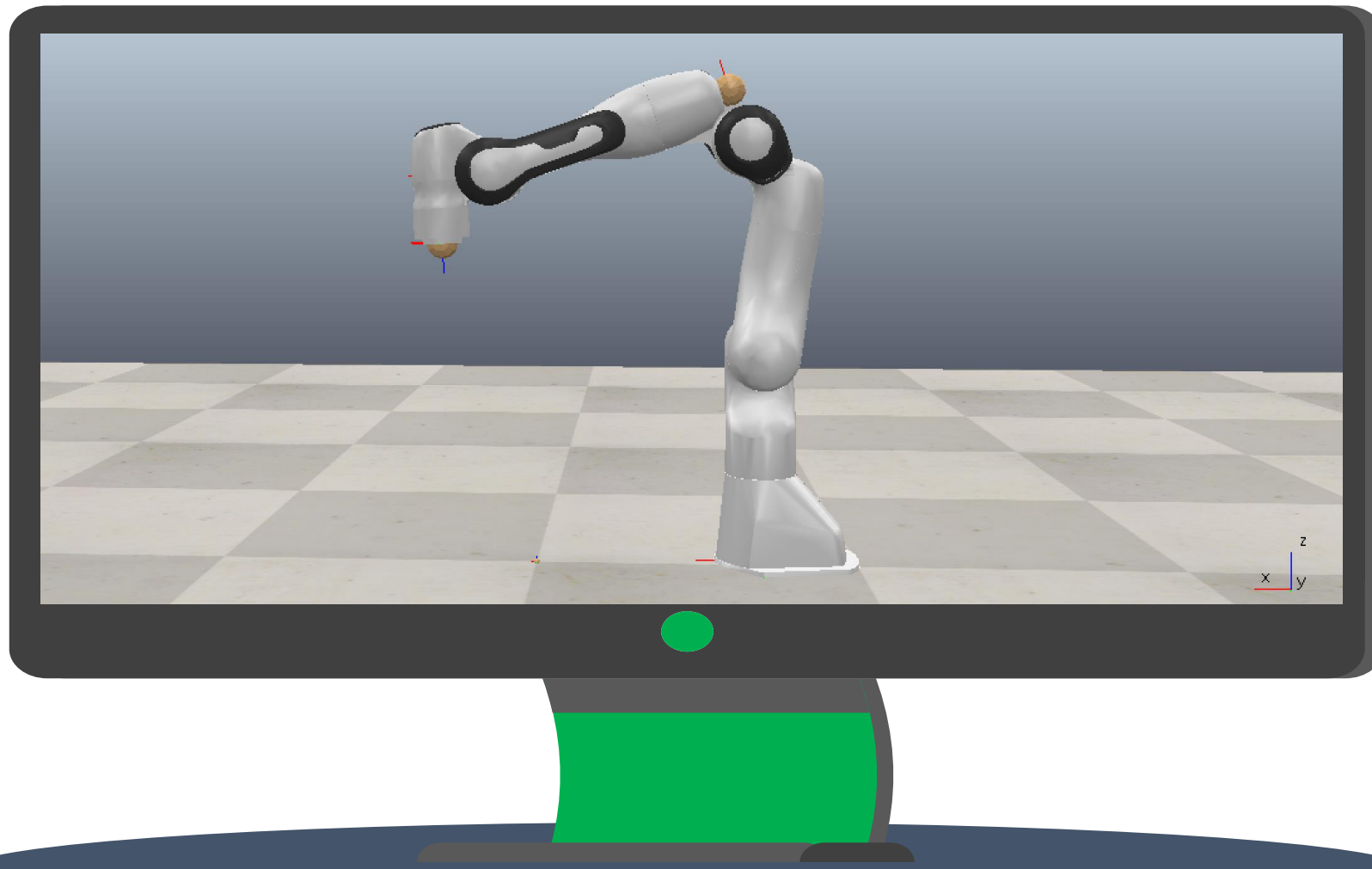
# Publish the joint velocity and all other topics required by the assignment
for i in range(0, self.dof):
    self.p_joint_cmd_vel[i].publish(q_dot[i])
    self.p_jacobian.publish(Jacobian(geo_jacobian))
```

Difficulties

- Lack of oversight
- Issues with end effector orientation
- Planned order of implementation
- Some uncertainty with the slides



Demo



Thank you for your attention

